

Implementationsdokumentation

Audio I/O & 3D-Raumdaten

Kais

5. Juli 2026

Inhaltsverzeichnis

Kapitel 1

Einführung

Dieses Dokument beschreibt die Implementierung zweier kritischer Komponenten des Distributed Acoustic Ray Tracers:

1. **Audio I/O (Ticket 3):** Einlesen trockener WAV-Dateien und Export des verarbeiteten, halligen Audio-Signals mit Clipping-Prävention
2. **Raumdaten (OBJ-Parser):** Laden von 3D-Modellen aus `.obj`-Dateien und Konvertierung in das interne JSON-Format

Die Implementierung erfolgt in Python und integriert sich nahtlos in die bestehende Pipeline, die Rust-basierte Physik-Engine, DSP-Convolution und Distributed-Computing-Frameworks verbindet.

1.1 Architektur-Übersicht

Das System folgt einer erweiterten Pipeline mit automatischer Raumgeometrie-Integration:

- [0/4] OBJ-Import: `room.obj` → `room_geometry.json` mit Material-Absorption
- [1/4] Rust Physics Engine: Lädt Raumgeometrie, simuliert Schallstrahlen, erzeugt IR
- [2/4] Audio-Einlesung: Lädt trockene WAV-Datei
- [3/4] DSP-Convolution: Wendet IR mit frequenzabhängiger Material-Dämpfung an
- [4/4] Audio-Export: Normalisiert und speichert Ergebnis

Das Wandmaterial wird in `input_config.json` konfiguriert und durchgehend durch alle Stufen propagiert: von der Geometrie-Absorption im Rust-Tracer bis zur oktavbandbasierten Frequenzformung in der Convolution.

Kapitel 2

Audio I/O Handling

2.1 Anforderungen (Akzeptanzkriterien)

1. Sauberes Einlesen der `unprocessed.wav` ohne Qualitätsverlust
2. Export der finalen WAV-Datei ohne Artefakte
3. Normalisierung, damit geschichtetes Audio beim Export nicht clippt

2.2 Theoretischer Hintergrund

2.2.1 WAV-Format und Audio-Digitalisierung

Das WAV-Format speichert Audiodaten als zeitdiskrete Samples mit konstanter Abtastfrequenz f_s (typisch 44.1 kHz oder 48 kHz):

$$x[n] = \int_{(n-0.5)/f_s}^{(n+0.5)/f_s} x_{\text{analog}}(t) dt$$

Jedes Sample liegt im Bereich $[-1.0, 1.0]$ (bei normalisierten 16/32-Bit PCM).

2.2.2 Clipping und Normalisierung

Wenn während der Convolution (Faltung) die Amplitude $|y[n]|$ über 1.0 steigt, tritt **Clipping** auf:

$$y_{\text{clipped}}[n] = \begin{cases} 1.0 & \text{wenn } y[n] > 1.0 \\ y[n] & \text{wenn } |y[n]| \leq 1.0 \\ -1.0 & \text{wenn } y[n] < -1.0 \end{cases}$$

Um dies zu vermeiden, normalisieren wir:

$$y_{\text{norm}}[n] = \frac{y[n]}{A_{\text{max}}} \quad \text{wobei} \quad A_{\text{max}} = \max_n |y[n]|$$

2.3 Implementierung: `wav_handler.py`

2.3.1 WAV-Einlesung

```

1 import numpy as np
2 import soundfile as sf
3
4 def read_wav(wav_path: str) -> tuple:
5     """
6     Reads a WAV file and returns the audio data as a NumPy array
7     along with the corresponding sample rate.
8     """
9     audio_data, sample_rate = sf.read(wav_path)
10    return audio_data, sample_rate

```

Funktionsweise:

- Die soundfile-Bibliothek liest WAV-Dateien robust
- audio_data ist ein NumPy-Array der Form (samples,) oder (samples, channels)
- sample_rate ist die Abtastfrequenz in Hz

2.3.2 WAV-Export mit Normalisierung

```

1 def write_wav(wav_path: str, audio_data, sample_rate: int) -> None:
2     """
3     Normalizes the audio data to prevent clipping and exports it to a WAV file.
4     """
5     max_amplitude = np.max(np.abs(audio_data))
6     if max_amplitude > 1.0:
7         audio_data = audio_data / max_amplitude
8     sf.write(wav_path, audio_data, sample_rate)

```

Algorithmus:

Algorithm 1 Normalisierter WAV-Export

```

 $A_{\max} \leftarrow \max_n |y[n]|$ 
if  $A_{\max} > 1.0$  then
     $y[n] \leftarrow y[n]/A_{\max}$  für alle  $n$ 
end if
Schreibe normalisierte Samples in WAV-Datei

```

2.3.3 Integration in die Pipeline

```

1 # main.py Schritt [2/4]: Audio laden
2 dry_audio, sample_rate = read_wav(input_wav_path)
3
4 # Schritt [3/4]: Convolution mit Material-Absorption
5 conv_material = ir_data.get("metadata", {}).get("wall_material")
6 wet_audio = convolve(dry_audio, sample_rate, ir_data, material=conv_material)
7
8 # Schritt [4/4]: Normalisieren und exportieren
9 write_wav(output_wav_path, wet_audio, sample_rate)

```

Der wall_material-Wert wird automatisch aus ir_output.json gelesen, das der Rust-Tracer nach der Simulation schreibt. Damit nutzen Tracer und Convolution stets dasselbe Material.

Kapitel 3

3D-Raumdaten: OBJ-Parser

3.1 Theoretischer Hintergrund

3.1.1 OBJ-Format

Das Wavefront .obj-Format ist ein textbasiertes 3D-Modellformat:

```
1 # Vertices (Ecken)
2 v 1.0 2.0 -1.5 # Vertex 1
3 v 2.0 2.0 -1.5 # Vertex 2
4 v 1.0 3.0 -1.5 # Vertex 3
5
6 # Faces (Dreiecke)
7 f 1/1/1 2/2/1 3/3/1
```

Indexierung: OBJ nutzt 1-basierte Indizes, Python 0-basierte – Umrechnung nötig.

3.1.2 Dreiecks-Repräsentation

$$\text{Triangle} = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{bmatrix}$$

3.2 Implementierung: obj_parser.py

3.2.1 OBJ-Loader

```
1 def load_obj_to_triangles(file_path: str) -> list:
2     vertices = []
3     triangles = []
4     path = Path(file_path)
5     if not path.exists():
6         raise FileNotFoundError(f"The 3D file '{file_path}' was not found.")
7     with path.open('r') as f:
8         for line in f:
9             line = line.strip()
10            if line.startswith('v '):
11                parts = line.split()
12                x, y, z = float(parts[1]), float(parts[2]), float(parts[3])
13                vertices.append([x, y, z])
14            elif line.startswith('f '):
15                parts = line.split()
16                v1_idx = int(parts[1].split('/')[0]) - 1
17                v2_idx = int(parts[2].split('/')[0]) - 1
```

```

18         v3_idx = int(parts[3].split('/')[0]) - 1
19         triangle = [vertices[v1_idx], vertices[v2_idx], vertices[v3_idx]]
20         triangles.append(triangle)
21     return triangles

```

3.2.2 Material-basierter Export: export_room_geometry()

Die neue Exportfunktion verbindet OBJ-Geometrie mit der Materialdatenbank (`materials.json`) und schreibt `room_geometry.json` für den Rust-Tracer. Jedes Dreieck erhält die mittlere Absorptionskonstante des gewählten Materials:

```

1  _MATERIALS_JSON = Path(__file__).resolve().parent.parent / "rust_service" / "materials
   .json"
2
3  def _mean_absorption(material: str) -> float:
4      """Mittlere Absorption aus materials.json fuer das gewaehlte Material."""
5      data = json.loads(_MATERIALS_JSON.read_text())
6      alphas = data["materials"][material]["absorption"]
7      return sum(alphas) / len(alphas)
8
9  def export_room_geometry(obj_path: str, material: str, out_path: str) -> None:
10     triangles_raw = load_obj_to_triangles(obj_path)
11     absorption = _mean_absorption(material)
12     triangles = [
13         {"v0": t[0], "v1": t[1], "v2": t[2], "absorption": absorption}
14         for t in triangles_raw
15     ]
16     out = {"triangles": triangles, "triangle_count": len(triangles)}
17     Path(out_path).write_text(json.dumps(out, indent=2))

```

3.2.3 Ausgabeformat: room_geometry.json

Der Rust-Tracer liest Dreiecke inkl. Material-Absorption aus dieser Datei:

```

1  {
2      "triangles": [
3          {"v0": [0.0, 0.0, 0.0], "v1": [12.0, 0.0, 0.0],
4           "v2": [12.0, 12.0, 0.0], "absorption": 0.033}
5      ],
6      "triangle_count": 12
7  }

```

Die `absorption`-Werte entsprechen dem arithmetischen Mittel aller 8 Oktavbänder aus `materials.json`. Die frequenzabhängige Verarbeitung findet ausschließlich in der Python-Convolution statt.

Kapitel 4

Integration und Testing

4.1 End-to-End-Pipeline

Die Pipeline verläuft nun in 5 Schritten. Schritt 0 ist vollautomatisch: `room.obj` wird erkannt, mit dem konfigurierten Material in `room_geometry.json` umgewandelt und dem Rust-Tracer bereitgestellt.

```
1 # Schritt 0: OBJ -> room_geometry.json (automatisch wenn room.obj vorhanden)
2 if Path("room.obj").exists():
3     export_room_geometry("room.obj", wall_material,
4                          "../rust_service/room_geometry.json")
5
6 # Schritt 1: Rust Physics Engine
7 ir_data = run_rust_and_load_json(rust_binary_path, json_path)
8
9 # Schritt 2: Audio laden
10 dry_audio, sample_rate = read_wav(input_wav_path)
11
12 # Schritt 3: Convolution mit Material-Absorption
13 conv_material = ir_data.get("metadata", {}).get("wall_material")
14 wet_audio = convolve(dry_audio, sample_rate, ir_data, material=conv_material)
15
16 # Schritt 4: Normalisieren und exportieren
17 write_wav(output_wav_path, wet_audio, sample_rate)
```

4.2 Material-Konfiguration

Das Wandmaterial wird in `rust_service/input_config.json` festgelegt:

```
1 {
2     "wall_material": "concrete",
3     "max_bounces": 100,
4     "rays_to_cast": 1000
5 }
```

Vom Rust-Tracer wird `wall_material` in `ir_output.json` exportiert und von der Python-Pipeline automatisch für die Convolution übernommen. Vegerfügbare Materialien (aus `materials.json`):

Material	Beschreibung	Mittl. Absorption
concrete	Beton (stark reflektierend)	0.033
drywall	Trockenbau (mäßig)	0.150
glass	Glas (stark reflektierend)	0.026
carpet	Teppich (stark absorbierend)	0.425
foam	Schaum (sehr stark absorbierend)	0.650

Tabelle 4.1: Verfügbare Materialien in `materials.json`

4.3 Abhängigkeiten

Modul	Bibliotheken	Version
<code>wav_handler.py</code>	soundfile, numpy	Any
<code>obj_parser.py</code>	pathlib, json	Built-in
<code>convolution.py</code>	scipy, numpy	Any
<code>materials.json</code>	— (Datenbasis)	—

Kapitel 5

Mathematische Modelle

5.1 Convolution im Frequenzraum

$$Y(f) = X(f) \cdot H(f), \quad y[n] = \mathcal{F}^{-1}\{Y(f)\}$$

5.2 Oktavband-basierte Material-Dämpfung

Der Rust-Tracer liefert pro Schallstrahl nur einen skalaren Druck. Die frequenzabhängige Dämpfung des Wandmaterials wird in der Convolution nachträglich im Frequenzbereich modelliert.

Ein Tap zur Zeit t hat circa $n(t) = t/t_{\text{mfp}}$ Reflexionen erfahren. Pro Oktavband b mit Absorptionskoeffizient α_b verbleibt der Anteil:

$$\text{Gain}_b(t) = (1 - \alpha_b)^{n(t)} = (1 - \alpha_b)^{t/t_{\text{mfp}}}$$

Die Impulsantwort wird bandweise gefiltert und mit dem zeitvarianten Gain gewichtet:

$$h_{\text{mat}}[n] = \sum_b \underbrace{(\text{Bandpass}_b * h)[n]}_{\text{gefilterte IR}} \cdot (1 - \alpha_b)^{n/(f_s \cdot t_{\text{mfp}})}$$

Hohe Bänder mit großem α_b (z.B. Teppich: $\alpha_{8\text{ kHz}} = 0.70$) klingen schnell aus — der Raum klingt dunkler und trockener. Bei Beton ($\alpha_b \approx 0.02\text{--}0.05$) bleibt der Klang lange hell.

5.3 Ray-Tracer Intersection (Möller-Trumbore)

Gegeben Strahl $\vec{r}(t) = \vec{o} + t\vec{d}$ und Dreieck $(\vec{a}, \vec{b}, \vec{c})$:

$$t = \frac{(\vec{c} - \vec{a}) \times (\vec{b} - \vec{a}) \cdot (\vec{a} - \vec{o})}{(\vec{c} - \vec{a}) \times (\vec{b} - \vec{a}) \cdot \vec{d}}, \quad \tau = \frac{t}{343 \text{ m/s}}, \quad n = \lfloor \tau \cdot f_s \rfloor$$

Kapitel 6

Akzeptanzkriterien und Validierung

6.1 Ticket 3: Audio I/O

Kriterium	Erfüllt?	Test
Sauberer Einlesen .wav	✓	<code>len(audio_data) > 0</code>
Export ohne Qualitätsverlust	✓	Hörtest, Spektrum-Analyse
Normalisierung gegen Clipping	✓	<code>max(abs(audio)) <= 1.0</code>

6.2 Raumdaten: OBJ-Parser & Material-Integration

Kriterium	Erfüllt?	Test
Parser liest .obj	✓	<code>len(triangles) > 0</code>
JSON-Export mit Absorption	✓	absorption-Feld pro Dreieck
Rust lädt OBJ-Geometrie	✓	<code>room_geometry.json</code> erkannt
Material durchgereicht	✓	<code>wall_material</code> in <code>ir_output.json</code>
Frequenzabhäng. Convolution	✓	<code>apply_material_absorption()</code>

Kapitel 7

Zusammenfassung und Ausblick

7.1 Implementierte Features

1. **Audio I/O:** Robust WAV-Lesen/Schreiben mit `soundfile`
2. **Clipping-Prävention:** Peak-Normalisierung vor Export
3. **OBJ-Parser:** Liest Vertices/Faces, konvertiert in Dreiecke
4. **Material-Integration:** `export_room_geometry()` verbindet OBJ mit `materials.json`
5. **Rust-OBJ-Pipeline:** Rust lädt `room_geometry.json` automatisch
6. **Material-Durchleitung:** `wall_material` von Config bis Convolution
7. **Frequenzabhängige Dämpfung:** Oktavband-EQ via `apply_material_absorption()`
8. **Pipeline-Integration:** Vollautomatisch ab Schritt 0 (OBJ-Erkennung)

7.2 Mögliche Erweiterungen

1. **Multi-Channel Audio:** Stereo/5.1 Surround Support
2. **Per-Dreieck-Materialien:** Verschiedene Materialien pro Wandfläche aus `.mtl`-Datei
3. **Adaptive Normalisierung:** Loudness-basierte Pegel (LUFS) statt Peak
4. **Geometrie-Optimierungen:** BVH-Tree für schnelle OBJ-Intersects

Anhang A

Code-Referenz

A.1 wav_handler.py

Code-Referenz: `python_service/wav_handler.py`

A.2 obj_parser.py

Code-Referenz: `python_service/obj_parser.py`

A.3 main.py

Code-Referenz: `python_service/main.py`